Smart Parking System: ParkSense

Project	Smart Parking System
Team name	MFRMYR
Milestone	M1 - Proposal and Architectural Requirements
Instructor / TA	Dr. Nada Mahmoud Hassan / Eng. Mohamed Fareh
Date	October 23, 2026

Team members

#	Name	Student ID
1	Farida Ahmed	231014770
2	Mira Atef	231005027
3	Rokaya Hamdy	231006586
4	Mohamed Shahin	231006169
5	Ronnie Sameh	231005375
6	Youssef Mahmoud	231006058

Contribution table

Member	Contribution
Ronnie Sameh	Section 1 (System overview), References
Mohamed Shahin	Section 2 (Stakeholders and Concerns)
Mira Atef	Section 3 (Context diagram) Section 4 (Functional requirements, MoSCoW, Prototype scope)
Farida Ahmed	Section 5 (Quality attribute scenarios)
Youssef Mahmoud	Section 6 (Constraints)
Rokaya Hamdy	Section 7 (Utility tree and ASRs)

Project Idea: Smart Parking System

The Smart Parking System is a platform that helps drivers find, reserve and pay for parking in busy urban and institutional areas, removing the need to circle looking for a free space. It supports three distinct user roles: drivers, who search for lots with live availability, reserve a spot, pay online and receive notifications; parking lot administrators and operators, who monitor occupancy on a web dashboard, configure pricing and manage lots; and IoT parking sensors, which send live occupancy updates.

The system integrates with external services, including a payment gateway, a navigation API and an SMS/notification gateway. It is split into independently deployable parts: a mobile/web client, an admin dashboard, a reservation and payment backend, and an event-ingestion service for sensor data. Its architecture is shaped by competing quality attributes. Strong consistency (never double-booking a spot) must be balanced against availability and fault tolerance during failures or network partitions, and real-time performance must be balanced against security (PCI-DSS-compliant payments).

The scope is realistic for a 5-person team. The prototype is a vertical slice covering search, reservation, sandbox payment, notifications and the operator dashboard, with sensors, gates and the payment gateway simulated.

1. System Overview

1.1 Problem Statement

Finding a parking spot in busy urban and institutional areas is a persistent challenge for drivers. Individuals often waste significant time and fuel driving in circles to locate vacant spaces, contributing directly to traffic congestion, heightened air emissions, and driver frustration. Traditional parking lots lack real-time monitoring infrastructure, meaning drivers only discover whether a spot is available after physically arriving at the location.

1.2 Proposed Solution & Target Users

The **Smart Parking System** is an enterprise-grade software solution engineered to streamline parking discovery, reservations, and management. The system supports three distinct user roles:

1. **Drivers/End Users:** Interact via a mobile or web client to view real-time parking availability, reserve spaces in advance, initiate digital payments, and receive navigation guidance.
2. **Parking Lot Administrators:** Monitor lot occupancy through a centralized web dashboard, configure pricing rates, manage physical spot mappings, and review operational audit logs and reports.
3. **IoT Sensor Devices & External Services:** Embedded parking spot sensors transmit live occupancy updates via lightweight IoT protocols, while external payment gateways and navigation APIs process financial transactions and routing.

1.3 Architectural Interest

This system presents compelling architectural challenges. It demands a hybrid architectural style that bridges high-throughput, low-latency IoT ingestion pipelines (handling real-time sensor updates across hundreds of concurrent slots) with a secure, transactional backend handling user reservations and financial clearances under strict PCI-DSS constraints. Key architectural trade-offs involve balancing real-time consistency (ensuring a reserved spot is never double-allocated) against system availability and fault tolerance during network partition events.

2. Stakeholders and Concerns

Stakeholder	Role/Responsibility	Key Concerns
Driver/End User	Uses the mobile or web application to view real-time parking availability, reserve a space, pay, and receive guidance.	Accurate and up-to-date availability; fast search and booking; easy-to-use interface; secure payments; reliable booking confirmations.
Parking Lot Administrator	Manages parking spaces, monitors occupancy, configures rates, and reviews operational reports.	Reliable dashboard; accurate sensor information; secure administrator access; efficient space and rate management; useful reports.
IoT Parking Sensors	Detects whether parking spaces are free or occupied and sends status updates to the system.	Accurate detection; stable connectivity; timely data transmission; handling disconnected or malfunctioning sensors.
Payment Gateway	Processes digital payments for parking reservations.	Secure transactions; reliable API integration; correct payment status; protection of payment data; compliance with applicable payment standards.
Navigation Service	Provides directions from the driver's location to the selected or reserved parking spot.	Accurate location and route information; dependable API connection; timely responses.
Notification Service/SMS Gateway	Sends booking confirmations, cancellation updates, payment-status alerts, and reminders through in-app notifications or SMS.	Reliable delivery; timely alerts; integration availability; handling third-party service failures and delivery delays.

3. System Context (C4 Level 1)

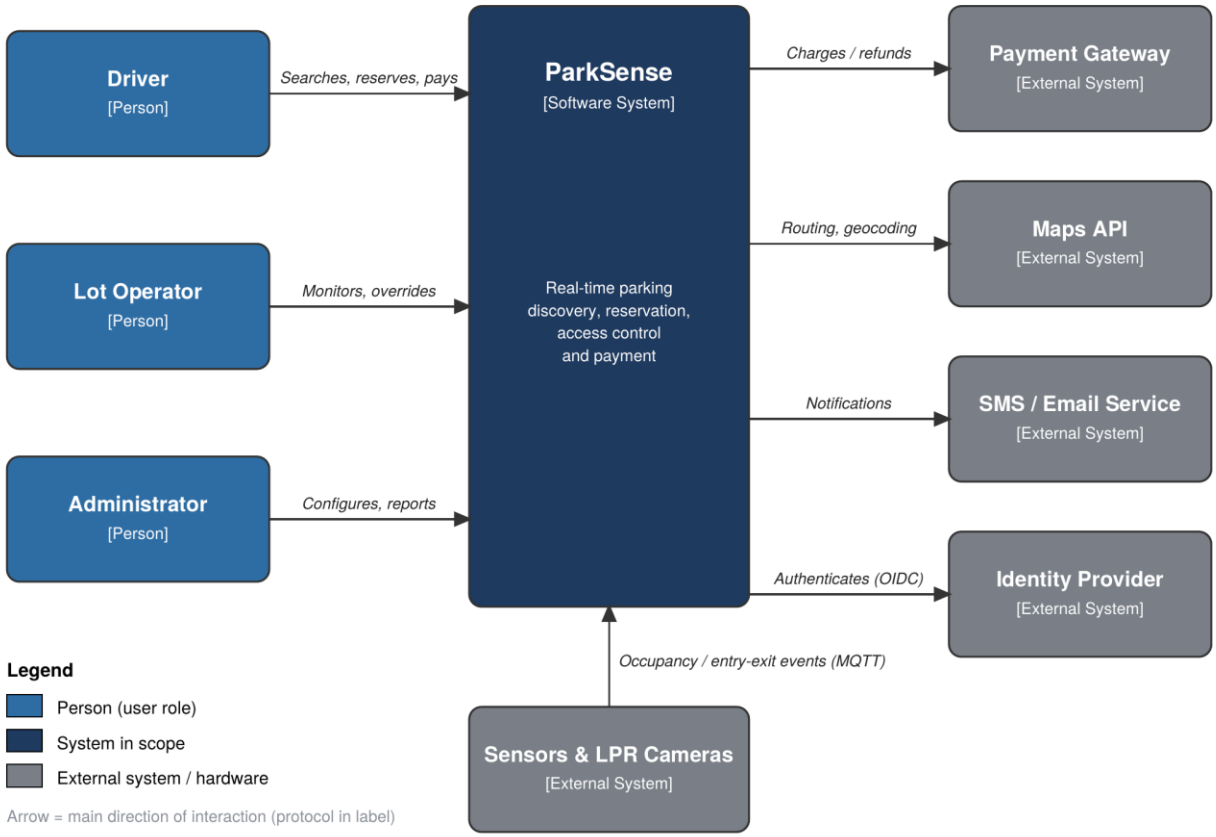


Figure 1 - C4 Level 1 context diagram of ParkSense.

4. Functional Requirements

There are 15 user stories, prioritized with MoSCoW (Must/Should/Could/Won't for this release).

- **Must (8):** login, search, reserve, pay, automatic gate, live occupancy, operator dashboard, admin management.
- **Should (4):** cancel/extend, notifications, navigation, reports.
- **Could (2):** dynamic pricing, subscriptions.
- **Won't (1):** EV charging and loyalty points.
- **Prototype:** a vertical slice of UC-01 to UC-08 and UC-10, which covers every architecturally significant requirement.

ID	User story	MoSCoW	Prototype (M3)
UC-01	As a driver, I want to log in, so my bookings are tied to me.	Must	Yes
UC-02	As a driver, I want to see nearby lots with live availability and price, so I can choose where to park.	Must	Yes
UC-03	As a driver, I want to reserve a spot, so it is guaranteed when I arrive.	Must	Yes

ID	User story	MoSCoW	Prototype (M3)
UC-04	As a driver, I want to cancel or extend a booking, so I only pay for what I need.	Should	Yes
UC-05	As a driver, I want to pay online, so I do not handle cash.	Must	Yes (sandbox)
UC-06	As a driver, I want the gate to open automatically (plate or QR), so entry and exit are fast.	Must	Yes (simulated)
UC-07	As the system, I want to update occupancy from sensor events, so availability is always live.	Must	Yes (simulated)
UC-08	As a driver, I want confirmations and reminders, so I do not miss my booking.	Should	Yes (email/log)
UC-09	As a driver, I want navigation to my lot, so I can find it easily.	Should	No (deep link only)
UC-10	As an operator, I want a live dashboard with manual override, so I can handle problems.	Must	Yes
UC-11	As an admin, I want to manage lots, pricing and users, so I can configure without code.	Must	Partial (lots, pricing)
UC-12	As an admin, I want occupancy and revenue reports, so I can track performance.	Should	No
UC-13	As the business, I want dynamic pricing, so we earn more at peak demand.	Could	No
UC-14	As a driver, I want a monthly permit, so I can park regularly with less hassle.	Could	No
UC-15	As a driver, I want EV charging and loyalty points, so I get extra perks.	Won't	No

Points to discuss

- **Core flow:** search (UC-02) → reserve (UC-03) → pay (UC-05) → gate opens (UC-06). Every Must story supports this flow.
- **Real time:** UC-07 feeds UC-02 and UC-10, so sensor events must reach the app and dashboard within seconds.
- **Simulated parts:** sensors, cameras and gates (UC-06, UC-07) and the payment gateway (UC-05) are simulated or sandboxed in the prototype.
- **Scope cut:** navigation is a deep link only (UC-09); reports (UC-12) and the Could/Won't stories are left out.

5. Quality Attribute Scenarios

Eight scenarios in the six-part form, covering seven quality attributes: performance, consistency/integrity, availability (two scenarios), security, modifiability, scalability and usability. Every response measure is testable.

QAS-1 - Live search (Performance)

Source	Driver using the mobile application
Stimulus	Searches for nearby parking lots
Artifact	Search and availability service
Environment	Normal operation, 500 concurrent users
Response	Returns a list of lots with live availability and price
Response measure	95% of searches finish in under 1 s and 99% in under 3 s, measured in a load test

QAS-2 - No double booking (Consistency/Integrity)

Source	Two drivers
Stimulus	Both try to reserve the last free spot at the same moment
Artifact	Reservation service and database
Environment	Normal operation, peak load
Response	Only one reservation is confirmed; the other driver is told the spot is taken and is shown other lots
Response measure	0 double bookings in a test of 1,000 simultaneous attempts on 100 spots; 95% of responses within 3 s

QAS-3 - Service failure (Availability)

Source	Infrastructure fault
Stimulus	One backend service instance crashes
Artifact	Reservation service
Environment	Peak hours, 500 concurrent users
Response	Requests move to a healthy instance, and the failure is logged and alerted
Response measure	Recovery within 30 s; 0 confirmed reservations lost; 99.5% monthly booking availability

QAS-4 - Sensor failure (Availability/Fault tolerance)

Source	IoT parking sensor
Stimulus	The sensor becomes Malfunctioning or Disconnected (T-4).
Artifact	Sensor service and availability data
Environment	Normal operation,500 concurrent users.
Response	The spot is flagged as unknown and cannot be booked, and the Administrator is alerted. Other spots keep updating, and the spot returns to normal when the sensor recovers.
Response measure	The spot is flagged within 30 s and the Administrator is alerted within 60 s. No bookings are accepted on a flagged spot, and other spots still update within 2 s (T-5).

QAS-5 - Access control (Security)

Source	Malicious user, or an attacker with a forged or expired token
Stimulus	Requests another user's bookings or payment records
Artifact	API gateway and authorization layer
Environment	Normal operation
Response	Request is rejected, logged and alerted; no card data is stored by the system (T-1)
Response measure	100% of 50 test requests return 401/403; log entry within 5 s; 0 card numbers found in a database scan

QAS-6 - Replace payment provider (Modifiability)

Source	Business owner
Stimulus	Requests switching to a different payment gateway
Artifact	Payment integration module
Environment	Development, before release
Response	A new adapter is written and configured; no other service changes (T-7)
Response measure	Done in 5 person-days or less; only the payment adapter changes; all regression tests pass

QAS-7 - City-wide growth (Scalability/Performance)

Source	IoT sensors in 100 lots
Stimulus	Update rate grows tenfold, to 2,000 events per second
Artifact	Event ingestion pipeline and availability data
Environment	Peak hour
Response	Ingestion scales by adding instances, with no redesign (T-9)
Response measure	95% of sensor updates reach the app within 2 s (T-5); 0 events lost; scaling needs configuration only

QAS-8 - First-time use (Usability)

Source	First-time driver
Stimulus	Wants to find a lot, reserve and pay
Artifact	Mobile and web application interface
Environment	Normal operation, no instructions given
Response	Completes the flow without help, with clear feedback at each step
Response measure	At least 9 of 10 test users succeed in 90 s or less, with at most 1 error

6. Constraints

Type	ID	Constraint
Technical	T-1	Must integrate with an external payment gateway (Visa, Mastercard, wallet) and must not store card data.
Technical	T-2	Must encrypt all user and payment data with AES-256 and use TLS for all traffic.
Technical	T-3	Must require 2FA for admin logins, with session timeout and role-based access control.
Technical	T-4	Must ingest spot status from IoT sensors and tolerate Working, Malfunctioning, and Disconnected states.
Technical	T-5	Must push availability updates to clients in under 2 seconds (e.g., WebSockets/SSE/MQTT).
Technical	T-6	Must support 500 concurrent users, with actions completing within 3 seconds.
Technical	T-7	Must integrate a payment gateway, maps API, and SMS/push service, without core booking depending on their uptime.
Technical	T-8	Must comply with data-protection law (e.g., GDPR) and show a privacy notice before registration.
Technical	T-9	Must allow new lots or cities to be added without code changes.
Technical	T-10	Must run on a free cloud tier, with daily backups.
Technical	T-11	Must be built as at least two independently deployable parts.
Business	B-1	Must not include sensor, camera, or wiring work; sensors are assumed to exist.
Business	B-2	Must exclude AI parking prediction, license-plate recognition, subscriptions, and citywide traffic integration from this release.
Business	B-3	Must keep rates and offers configurable, since lot owners set them.
Organizational	O-1	Must be delivered by a six student team.
Organizational	O-2	All work submitted via GitHub; architecture documented with C4 / 4+1 views and ADRs; diagram sources kept in the repository.
Legal/Regulatory	L-1	Comply with Egypt's Personal Data Protection Law (No. 151 of 2020): minimal collection of location and plate data, consent, retention limits.

7. Architecturally Significant Requirements (ASRs)

7.1 Utility tree

Each leaf is a scenario rated (Business value, Architectural Risk/Difficulty) as High/Medium/Low.

Quality attribute	Scenario	(Value, Risk/Difficulty)
Consistency/Integrity	QAS-2: No double booking under concurrent attempts	(H, H)
Availability	QAS-4: Detect a malfunctioning or disconnected sensor within 30 seconds, flag its spot as unknown, and alert the administrator within 60 seconds, while other spots continue updating within 2 seconds under 500 concurrent users.	(H, H)
Availability	QAS-3: Cloud failover within 30 seconds, with 99.9% monthly availability	(H, M)
Scalability/Performance	QAS-7: Process 2,000 events/s, with 95% visible within 3 seconds	(H, H)
Performance	QAS-1: 95% of requests completed within 500 ms at 500 concurrent users	(M, M)
Security	QAS-5: Deny 100% of unauthorized requests and store no card data	(H, H)
Modifiability	QAS-6: Replace payment provider within 5 person-days	(M, H)
Usability	QAS-8: 9 out of 10 users reserve and pay within 90 seconds	(M, L)

7.2 Prioritized requirements

Rank	Requirement	Related Scenario	Business value	Architectural impact	ASR?
1	Prevent double booking	QAS-2	H	H	Yes
2	Sensor failure detection and fault tolerance	QAS-4	H	H	Yes
3	Real-time sensor event processing at scale	QAS-7	H	H	Yes
4	Security and data protection	QAS-5	H	H	Yes
5	Replaceable external integrations	QAS-6	M	H	Yes
6	Cloud failover and service availability	QAS-3	H	M	No
7	Search response time	QAS-1	M	M	No
8	Reservation and payment usability	QAS-8	M	L	No
9	Supporting functional features	UC-09,UC-12 to UC-15	L	L	No

7.3 Justification of the top five ASRs

ASR-1 No double booking (QAS-2):

- It's the central promise of a reservation system: if a booking is confirmed, the spot should be guaranteed.
- Incorrect bookings can cause disputes, inconvenience, refunds, and loss of trust.
- Concurrent requests can arrive at the same time, so simply checking whether a spot is available before saving the reservation isn't enough.
- The architecture must define how conflicting requests are handled, such as through database transactions, locking, or another concurrency-control strategy.

ASR-2 Sensor Failure Detection and Fault Tolerance (QAS-4):

- A malfunctioning or disconnected sensor can provide inaccurate or outdated parking availability information.
- Affected parking spots must be flagged as unknown within 30 seconds and made unavailable for new bookings to prevent reservations based on unreliable information.
- The administrator must be alerted within 60 seconds so that the sensor can be inspected and restored.
- The failure of one sensor should not interrupt the operation of other parking spots, which must continue updating within 2 seconds.
- The architecture must support sensor health monitoring, failure detection, safe handling of uncertain availability, administrator notifications, and recovery when the sensor becomes operational again.

ASR-3 Real-time ingestion at scale (QAS-7):

- Outdated availability could cause drivers to travel to lots that are already full.
- Multiple parking lots can generate many sensor events simultaneously.
- As the system grows, the volume of incoming events increases.
- Delayed or lost events can make the information shown to users unreliable.

ASR-4 Security and data protection (QAS-5):

- A driver must not be able to view another driver's private bookings or payment records.
- Administrators need stronger access controls because they can manage parking lots, users, and prices.
- Payment information and personal data require careful protection.
- A security breach can damage user trust and create legal or financial consequences.

ASR-5 Replaceable integrations (QAS-6):

- Changing a payment provider shouldn't require rewriting the entire reservation system.
- A third-party service can fail even when your own application is working correctly.
- The project needs to keep its core booking logic independent of particular providers.
- Using well-defined interfaces and an interchangeable integration design can reduce the effort required to replace external services.

References

[1] L. Bass, P. Clements and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA, USA: Addison-Wesley, 2021.

[2] S. Brown, "The C4 model for visualising software architecture," c4model.com. [Online]. Available: https://c4model.com

[3] P. Kruchten, "The 4+1 view model of architecture," IEEE Software, vol. 12, no. 6, pp. 42-50, Nov. 1995.

[4] ISO/IEC 25010:2011, Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models.

[5] Arab Republic of Egypt, Personal Data Protection Law No. 151 of 2020.

[6] R. Kazman, M. Klein and P. Clements, "ATAM: Method for architecture evaluation," CMU/SEI-2000-TR-004, Software Engineering Institute, 2000.